Python Debugging — Worked Example: Solution

> Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Hints

Story — `python_debugging_practice.py`: each round's buggy function, its fix, and the test that would have caught the bug ahead of time all go into this one file, so you can actually run the bug, watch it fail with a real traceback, fix it, then run the test that would have caught it — the whole debugging method, practiced end to end, in one place. **If not:** you'd just read the answer, nod along, and never find out whether you could have actually spotted the bug yourself.

Every fix and test below goes in `practice/python_debugging_practice.py`, and runs with `pytest python_debugging_practice.py -v` from inside `practice/`.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent
- Real cause vs. symptom fix

Round: Basic

Real cause: `trim_history()` never has a `return` statement on the path where `len(trimmed) % 2 != 0` is false — it just falls off the end of the function, and a function with no `return` always gives back `None`.

Story: the crash line itself (`for msg in ...`) is one function call away from where the actual bug lives, so this round trains the most basic trace-reading move: "something returned `None` where a list was expected — go to the function that returned it and check *every* code path actually returns something." **If not:** you'd stare at the loop that crashed, which looks fine on its own, and never look at the function that fed it bad data.

The fix:

```
def trim_history(history, max_turns):
    trimmed = history[-max_turns:]
    if len(trimmed) % 2 != 0:
        trimmed = trimmed[1:]
    # why: without this line the function falls off the end,
```

```
    # and every Python function that does that returns None
    return trimmed
```

Test that would have caught it:

```
def test_trim_history_returns_a_list():
    history = ["a", "b", "c", "d"]
    result = trim_history(history, 4)
    assert result is not None
    assert isinstance(result, list)
```

Reading the trace from the bottom up: the error fires at the `for msg in ...` line, but the actual bug is one function call away — the fastest read is "something is returning `None`," then going straight to that function and checking every branch has a `return`.

Round: Intermediate

Real cause: `notes=[]` is a **mutable default argument**. Python creates that empty list exactly once, when the function is defined — not once per call. Every call to `log_turn()` that doesn't pass its own `notes` argument gets the *same* list object back, and keeps appending to it, for as long as the process stays alive. It looks like a session/state bug because the symptom is "data leaking between sessions" — but no session code is involved at all; it's one shared list living on the function object itself.

Story: the symptom here looks exactly like it points at session code — but no session code is actually involved in the bug at all. This round trains checking the plain Python layer first (argument defaults, variable scope) before assuming the bug lives in the bigger, more complex system. **If not:** you'd spend hours investigating session isolation and never think to look at a simple function signature.

The fix:

```
def log_turn(role, text, notes=None):
    # why: a default list is made ONCE, when the function is defined,
    # and shared by every call — None plus a new list per call fixes it
    if notes is None:
        notes = []
    notes.append(role + ": " + text[:40])
    return notes
```

Using `None` as the default, and creating a fresh list inside the function body, means every call that doesn't pass its own list gets a brand new one.

Test that would have caught it:

```
def test_log_turn_does_not_leak_between_calls():
    first_session = log_turn("user", "what is the refund policy?")
    second_session = log_turn("user", "how do I reset my password?")
    assert len(second_session) == 1
    assert "refund" not in second_session[0]
```

This test calls the function twice in a row, on purpose, without passing `notes` — exactly the condition that never shows up if every test only ever calls a function once.

Round: Real-world

Real cause: `for key in cache:` iterates directly over the dictionary while `del cache[key]` changes its size inside that same loop. Python detects this and raises `RuntimeError` — but only once a delete actually happens mid-loop. A cache with nothing expired yet just loops harmlessly to the end, which is exactly why quick manual testing (fresh cache, nothing expired) never triggers it.

Story: a bug that only shows up once real data has had time to age is the classic "works in every test I ran" failure — the test data never got old enough to hit it. The fix itself is simple; the real lesson is building a test whose data is deliberately shaped to actually trigger the bad path, not just the happy one. **If not:** your tests would keep using a fresh cache and keep passing comfortably, while production kept crashing on data your tests never resembled.

The fix:

```
def clear_expired(cache, now):
    # how: first pass only READS the dict and collects keys to delete
    expired_keys = []
    for key in cache:
        if cache[key]["expires_at"] < now:
            expired_keys.append(key)

    # how: second pass deletes — the first loop over cache is finished
    for key in expired_keys:
        del cache[key]
```

Collect the keys to remove in a first pass, then delete them in a second pass, after the first loop over `cache` has already finished.

Test that would have caught it:

```
def test_clear_expired_removes_entries_without_crashing():
    cache = {
        "a": {"expires_at": 1},
        "b": {"expires_at": 100},
        "c": {"expires_at": 1},
    }
    clear_expired(cache, now=50)
    assert "a" not in cache
    assert "c" not in cache
    assert "b" in cache
```

The test that "passed" during development only ever ran against an empty or all-fresh cache — the real test has to include at least one entry that's actually supposed to be deleted, otherwise the buggy loop never gets exercised.

Round: Multi-agent

Real cause: the exact same mutable-default-argument bug as Round 2, just in a place where the usual test (call the node once, check the `Command`) can't see it. `test_route()` only calls `route(state)` a single time before the test process exits, so the shared default list only ever has one entry in it when the assertion runs — it looks correct. The bug only appears once the *same process* handles several requests back to back, which a live multi-agent run does constantly and a narrow unit test never does.

Story: this is the same shape of bug as Round 2, now hidden inside a graph node, where a unit test that calls the node only once can never see it fire. This round trains writing a test that calls the node more than once in sequence, the way a real live pipeline would, which is the only way this particular bug shows up. **If not:** `test_route()` would stay green forever while the Reviewer kept rejecting confused drafts in production.

The fix:

```
from langgraph.types import Command

def pick_agent(state):
    # stand-in for Project 4's real routing choice
    return "research_agent"

def route(state, notes=None):
    # why: same fix as Round 2 — a fresh list on every call
    if notes is None:
        notes = []
    notes.append(state["task"])
```

```
    next_agent = pick_agent(state)
    return Command(goto=next_agent, update={"routing_notes": notes})
```

Test that would have caught it:

```
def test_route_does_not_leak_notes_across_requests():
    state_one = {"task": "summarize Q1 sales report"}
    state_two = {"task": "write a release blog post"}

    route(state_one)
    result_two = route(state_two)

    routing_notes = result_two.update["routing_notes"]
    assert len(routing_notes) == 1
    assert routing_notes[0] == "write a release blog post"
```

Same fix as Round 2, but the test that catches it has to call the node function more than once in the same process — a single-call unit test structurally cannot catch this bug, no matter how carefully it checks its one result.

Real cause vs. symptom fix

A symptom fix for Round 2 or 4 would be wrapping the caller in a `try/except` that clears `notes` whenever it looks "too long," or restarting the process between sessions so the shared list never has time to grow. Both would make the immediate failure go away without touching the actual bug — the shared default list would still be there, waiting to cause the same failure the next time the workaround doesn't quite cover the case. The real-cause fix is always the same one line, `if notes is None: notes = []`, because the real cause is always the same thing: a default argument evaluated once at definition time, silently shared by every call that doesn't override it. This is Core Concepts' "real cause vs. symptom fix" distinction directly — the fix that's actually correct is the one that would still work if you didn't know exactly when or how often the bug shows up.